BST kursa projekts
Gudrie radiatori

Ričards Bubišs rb22033

Markuss Birznieks mb20075

Motivācija

Agri no rīta guļot gultā, kad istabā ir vēss, negribas celties un regulēt radiatoru, būtu ērtāk ja varētu regulēt to no gultas. Pat vēl labāk, ja radiators sagatavotu no rītiem istabas temperatūru siltāk, bet tik tālu netikām. Ar gudriem radiatoriem arī ir devums dabai, kontrolējot nepieciešamo temperatūru telpā, piemēram, mazinot temperatūru, kad neviens nav mājās. Kā arī ar automātiski kontrolējamo temperatūru var mazināt pelējuma daudzumu pagrabtelpās, samazinot gaisa mitrumu.

Kas ir jau izdarīts pasaulē

Eksistē vairāki esoši risinājumi. Gudrie vārsti no AVATTO[1] un tado[2], kas ir kontrolējami attālināti, ir ar programmējamiem grafikiem un komunikācijai pielieto zigbee. Eksistē arī gudri pārnēsājami radiatori ar attālinātu kontroli un programmējamiem grafikiem. Alternatīvas atbalsta duālo temperatūras regulēšanu, izmatojot gan grafisko saskarni, gan manuālo termostata griešanu ar rokas palīdzību.

Pieeja un arhitektūra

Sistēma sastāv no vismaz 3 ESP motēm. ESP32 priekš centrālā servera kas ar ESP-NOW dod komandas ESP32-C3 motēm, kas kontrolē radiatorus. Kā arī ESP8266, kas hostē tīmekļa lietotni.

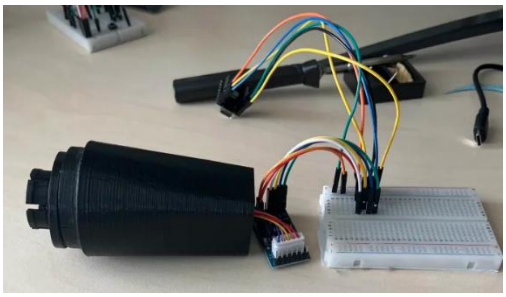
Radiatora kontrolieris ESP32-C3

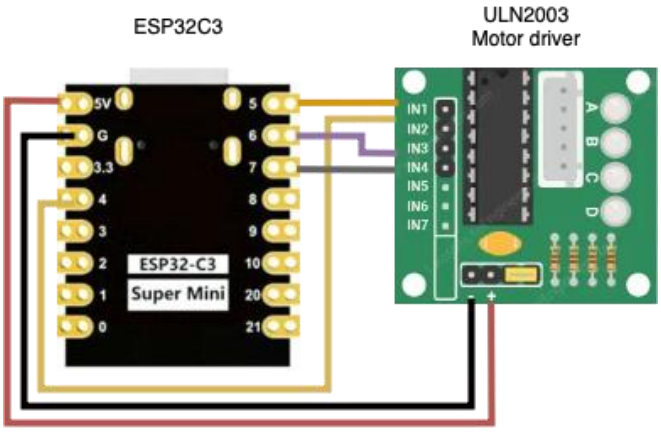
Radiatora kontrolieris kontrolē radiatoru ar soļmotoru spiežot virsū radiatora vārstam. Mehānisms satur no Lead Screw, kas pārvērš rotācijas spēku lineārā ar kura palīdzību mēs spiežam uz puļķi, kas regulē siltā ūdens plūsmas intensitāti radiatorā.



Radiatora kontroliera detaļas:

- 28BYJ-48 5V soļmotors
- ESP32-C3
- Mehānisms spiediena rašanai

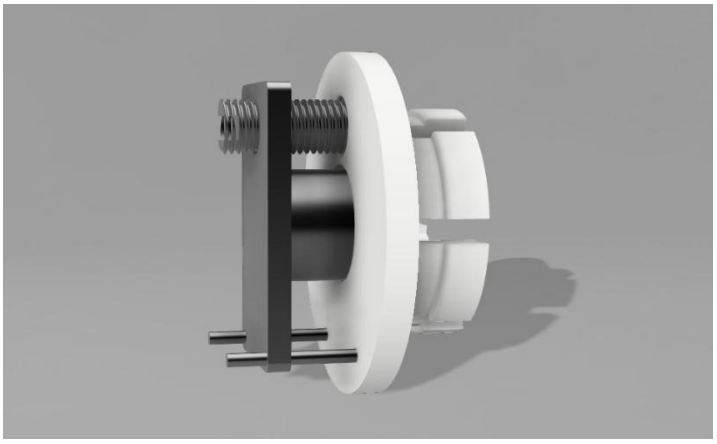




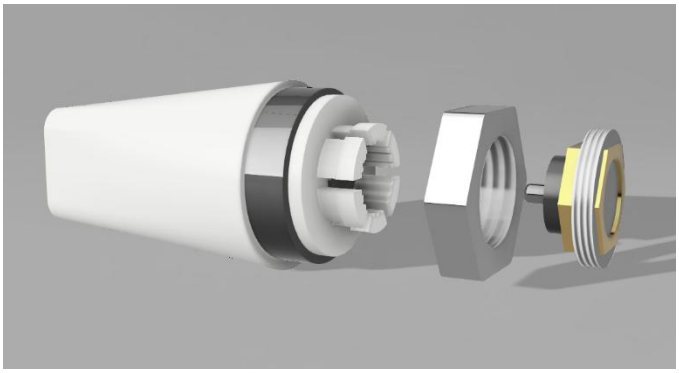
ESP uztver servera sūtīto uzstādāmo temperatūru un atbilstoši pagriež motoru. Pēc motora pagriešanas tas serverim sūta atbildi par veiksmīgi uzstādītu temperatūru.

Radiatora regulēšanas mehānisms

Radiatora regulēšanai tiek izmantots Lead Screw mehānisms, kas pārveido soļmotora rotācijas lineārās kustībās tādā veidā spiežot vairāk vai mazāk uz radiatora vārstu, laižot vairāk vai mazāk silto ūdeni radiatorā.







Precīzai temperatūras uzstādīšanai nepieciešama rūpīga kalibrācija. Nepieciešams noteikt soļu motora soļu intervālu temperatūras intervālam no 8-28 grādiem(standarta temperatūra visiem termostatiem).

Centrālais serveris

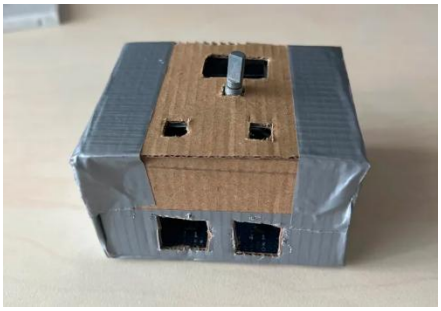
Centrālā servera uzdevums ir nodrošināt fizisku un tīmekļa saskarni temperatūras kontrolei. Tas uztver lietotāja temperatūras uzstādīšanu no rotary encoder vai tīmekļa lietotāja saskarnes un sūta temperatūras uzstādīšanas komandas attiecīgajiem radiatoriem (visiem vai specifiskam). Centrālais serveris sastāv no divām ESP motēm: ESP32 un ESP8266:

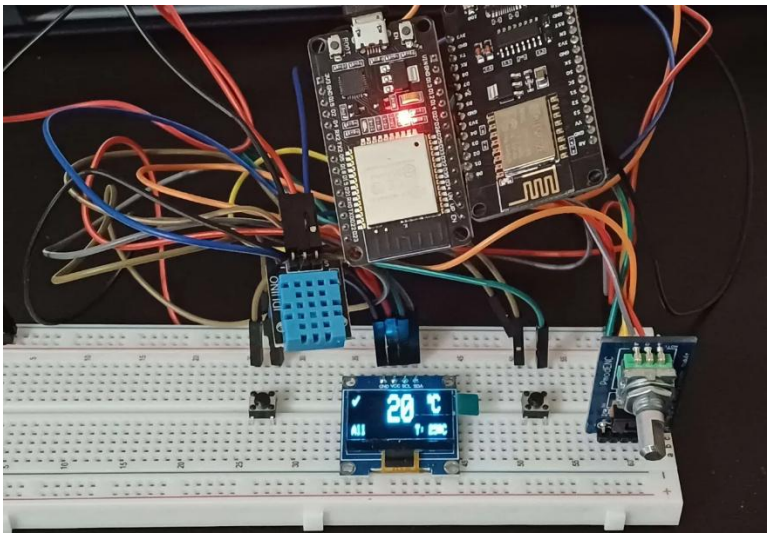
- ESP32 realizē fizisko saskarni ar pogām un ekrānu kā arī komunikāciju un komandu došanu radiatoru motēm (turpmāk ESP-Server).
- ESP8266 realizē tīmekļa serveri nodrošinot tīmekļa saskarni lietotājam(turpmāk ESP-Web).

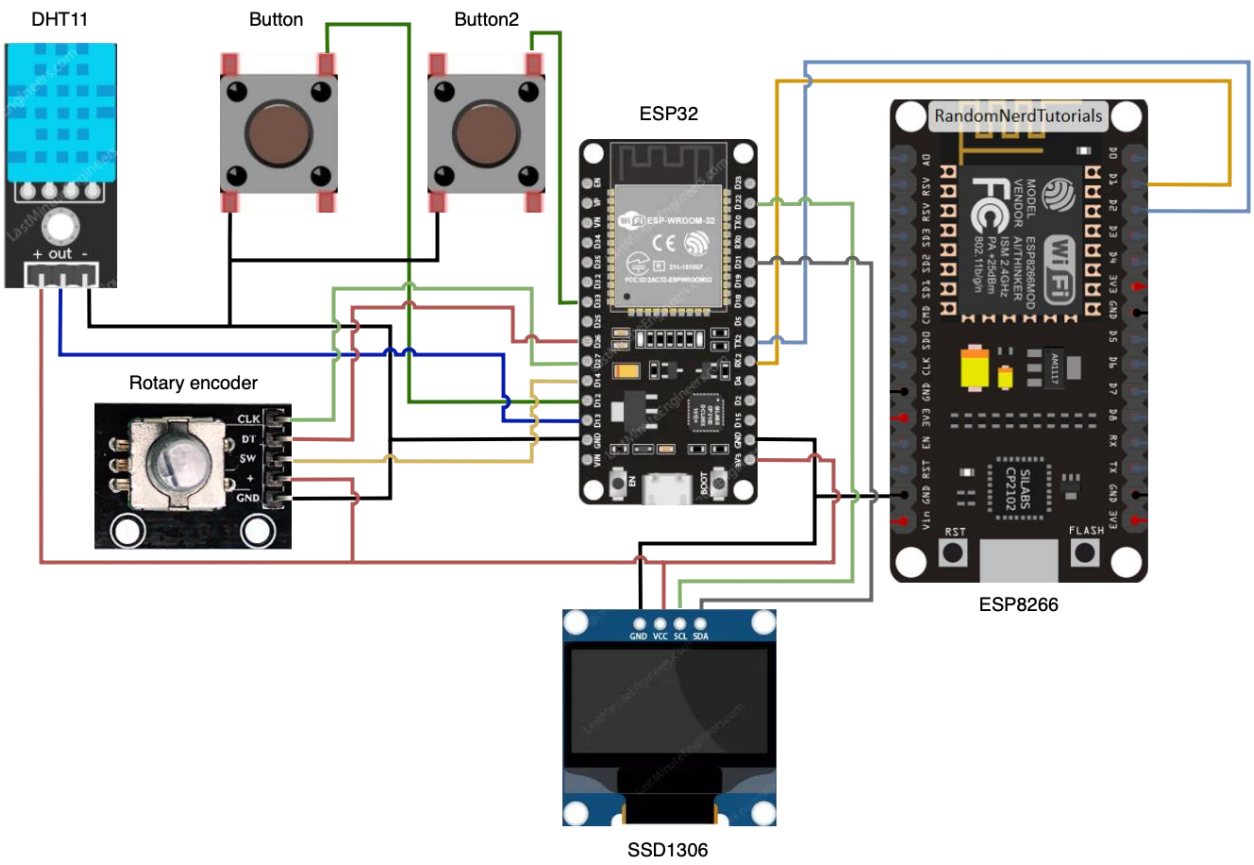
Starp ESP-Server un ESP-Web ir vada savienojums, lai nodrošinātu komunikāciju. Tie izmanto seriālo portu komunikāciju UART, kas nodrošina augstu piegādes rezultativitāti. Centrālai mezgls sūta informāciju serverim JSON formātā, lai to atjaunot tīmekļa vietnē. Serveris sūta ziņas plain teksta formātā, kas atbilst vienam no iepriekš definētiem formatējumiem, piemēram, SET-TEMP/ALL.

Centrālā servera detaļas:

- ESP32 priekš ESP-Server.
- ESP8266 priekš ESP-Web.
- DHT11 – mēra temperatūru.
- Rotary Encoder – lai uzstādītu temperatūru.
- Oled ekrāns.
- 2 pogas – viena lai mainītu izvēlēto radiatoru un otra lai apskatītu tīmekļa lietotnes pieslēguma datus.







Komunikācijas protokols

Komunikācijai starp iekārtām izmanto ESP-NOW, kas ļauj ESP iekārtām savā starpā komunicēt. Ziņojumi tiek sūtīti ar ESP funkciju esp_now_send(), norādot adresāta MAC adresi, datus un datu garumu. ESP funkcija atbild par to ka ziņojumu saņem norādītā MAC adrese. Datos tiek iekodēti dažādi ziņojuma tipi. Ziņojuma datos tiek apvienota galvene(header) un sūtāmie dati(payload).

Ziņojuma galvene norāda datu tipu, garumu kā arī maģisko simbolu virkni minimālai drošībai.

```
typedef struct {
  uint16_t magic;
  uint8_t type;
  uint8_t length;
} MessageHeader;
```

Galvene tiek apvienota ar sūtāmajiem datiem un nosūtīti ar ESP sūtīšanas funkciju.

```
  uint8_t buffer[250];
  MessageHeader header = { MESSAGE_MAGIC, type, length };

  memcpy(buffer, &header, sizeof(header));
```

```
  memcpy(buffer + sizeof(header), payload, length);

  esp_now_send(addr, buffer, sizeof(header) + length);
```

Šāda pieeja ļauj definēt dažādus ziņojumu tipus, nemainot sūtīšanas loģiku. Katram ziņojumu tipam ir sava definēta struktūra, lai tos būtu viegli atkārtoti izveidot un nolasīt saņemot. Viena no struktūrām tiek izmantota atklāšanas mehānismam.

Atklāšanas mehānisms

```
struct DiscoveryPayload {
  char name[MAX_NAME_LEN]; // identifies device type: “server” or “radiator”
  bool isResponse; // true if this is a reply
};
```

Atklāšanas mehānismā motes sūta tīklā DiscoveryPayload norādot broadcast adresi (0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF). Motei saņemot DiscoveryPayload tā reģistrē sūtītāja MAC adresi atmiņā un sūtītājam sūta atpakaļ atbildi, tā nodrošinot ka divas motes atklāj viena otru.

Vada komunikācija(UART)

Vada komunikācijai tiek izmantots komunikācijas protokols līdzīgs MQTT. ESP-Server lasa sūtītās rindiņas no ESP-Web un uztver tās kā komandas. Sūtītās rindiņas ir formātā, piemēram: “SET/TEMP/1/23”, “ALL/T/23”, “GET/RADIATORS”. Toties ESP-Server sūta ESP-Web JSON formāta datus par radiatoriem pēc tam kad ESP-Web nosūta “GET/RADIATORS”.

Tīmekļa lietotne

Tīmekļa vietnes hostošanai nepieciešams html paplašinājuma fails, kas satur informāciju par tīmekļa vietni. Izmantojot tādu pieeju vietnes satura izmērs ir ierobežots, jo programmas koda mainīgie tiek glabāti STACK atmiņā, kas visbiežāk nav pārāk liela. Domājot par problēmas risinājumu, atradām ērtu risinājumu, izmantojot LittleFS paplašinājumu, kas dod iespēju pārvaldīt mikrokontroliera FLASH jeb zibatmiņu, veidojot direktorijas un failus. LittleFS - “A little fail-safe filesystem designed for microcontrollers.” Ar failu sistēma palīdzību, mēs varam sadalīt tīmekļa vietnes saturu vairākos failos, atdalot frontend un backend funkcionalitāti, kā arī izmantot attēlus vai ikonas.

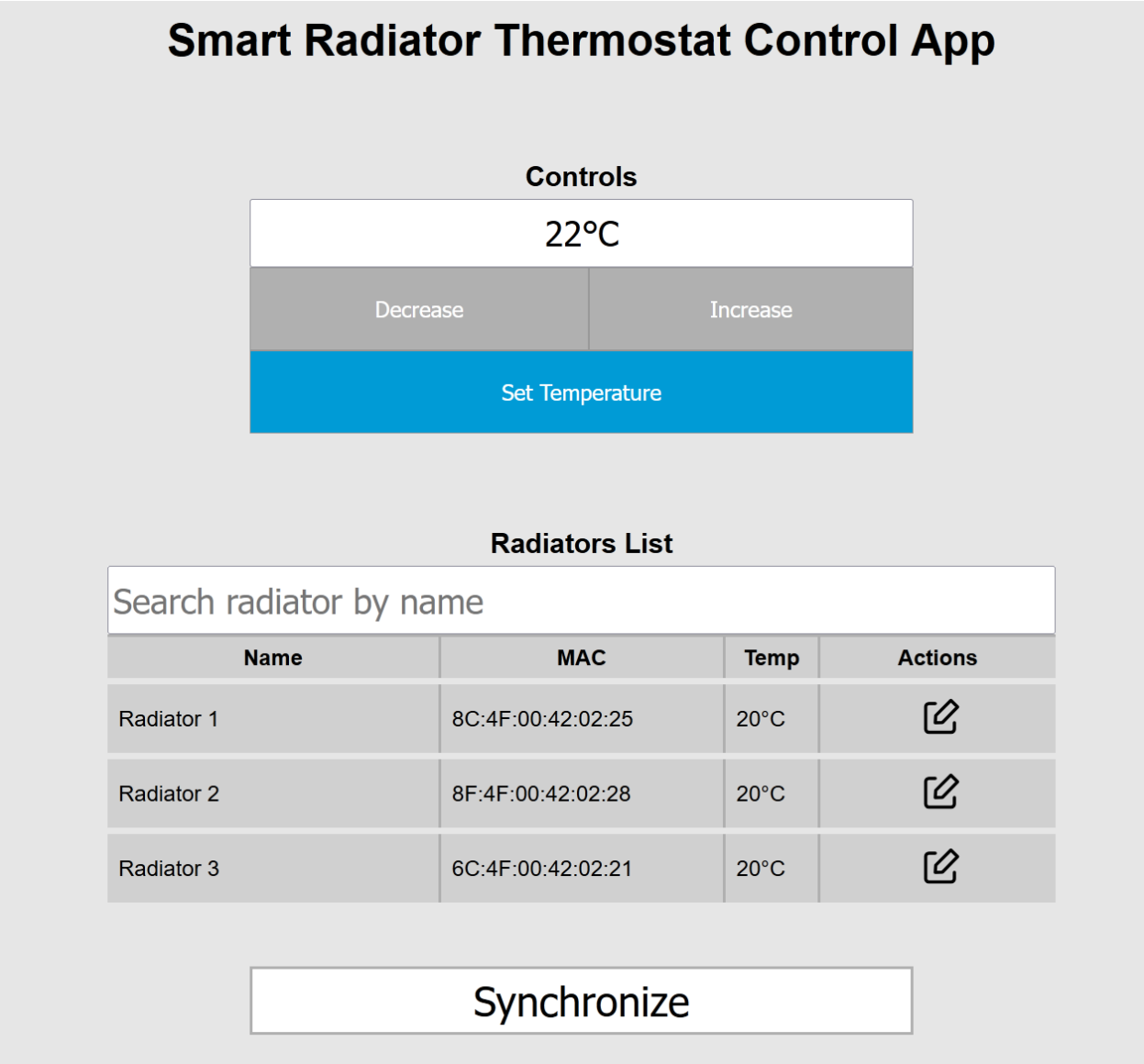
Tīmekļa lietotne tiek hostēta uz ESP8266 motes. Ir divi veidi kā hostēt tīmekļa lietotni:

1. Pieslēdzot moti mājas rūterim – tīmekļa lietotne būs pieejama mājas rūtera tīklā.

2. Darbinot moti kā rūteri (Access Point) – lietotājs var pieslēgties motei kā jebkuram rūterim, lai iegūtu pieeju tīmekļa lietotnei.



Lai nodrošinātu dinamisku darbību tiek izmantots WebSockets. Piemēram, mainot temperatūru tīmekļa lietotnē, tiek izsaukts galapunkts “/set”, un ESP-Web (ESP8266) reaģē atbilstoši uz izsaukumu.



Enerģija

Šobrīd mūsu risinājumā nav iekļauta nekāda enerģijas padeve no akumulatoriem, izmantojām tikai USB pieslēgumu. Mūsuprāt, centrālā mote varētu strādāt no kontaktlidzdas tīkla ar vada pieslēgumu, taču kontrolierim kas tiek uzstādīts radiatora termostata vietā jāspēj strādāt bez vadiem un pietiekami ilgi. Negribētos ik pēc dienas mainīt baterijas vai uzlādēt akumulatorus. Tika veikti nelieli aprēķini ar iespējamo 3000mAh akumulatora kapacitāti. Tika sarēķināts vidējais patēriņš motei un motoram ampēros, kas ir aptuveni ~290mA. Nepieciešamais spriegums 5V motoram un 3.3V motei.

Aprēķini:

3000mAh/290mA = ~10.3h

10.3h/0.5min(apmēram 30sekundes vienam darbības ciklam) = ~1200 cikli

Pieņemsim ka dienā temperatūra tiek mainīta 5 reizes:

1200/5 = ~240 dienas

Varētu teikt ka uz viena 3000mAh akumulatora radiatora viedais termostats varētu nostrādāt veselu apkures sezonu bez iejaukšanās.

Rezultāts

Kopumā mums izdevās izveidot nelielu mājas sensoru tīklu, kas spēj kontrolēt vairākus radiatorus no centrālā mezgla. Risinājumā izmantojām 2 motes centrālā mezgla vajadzībām, viens uztur fizisko ievadi un saziņu ar pārējiem radiatoru mezgliem, otrs uztur lietotāja saskarni tīmekļa vietnes veidā, atvieglojot temperatūras uzstādīšanu. Serveris un centrālā mote saziņai izmanto vadu savienojumu jeb UART protokolu, kas nodrošina iespēju komunicēt, izmantojot seriālos portus. Izmantojām 2 motes, lai atdalīt ievades, lai visu nedara viena mote.

Pirmajos risinājumos izmantojām cieti iekodētus radiatoru mezglus, taču turpmāk ieviesām dinamisko mezglu atklāšanu. Dabūt lai pēc ievades kustās motors bija vienkārši, bet nākamais solis izdarīt, lai tā motora rotācijas kustība tiktu pārveidota par lineāru spiešanas spēku. Nācās pētīt mehāniku un skatīt vairākus informācijas avotus. Tika lemts, ka vienkāršākais veids kā to izdarīt ir Lead Screw, kur uz skrūves ir objekts, kas ir nostiprināts pret rotāciju, un tad galvenā skrūve uz kuras ir objekts tiek rotēta. Rotācijas rezultātā objekts spēj pārvietoties abos virzienos pa skrūves vītni. Izmantojot Fusion 360 tika uzmodēlēta pirmā fiziskā modeļa versija, kas bija diezgan sarežģīti, jo pieredzes darbā ar to nebija.

Rakstot sistēmas kodu tika rūpīga plānots un dalīts kods, lai to būtu viegli pārskatīt un funkcionalitātes būtu loģiski atdalītas, sadalot kodu klasēs (Communications, RadiatorManager, WebComs), failos un 3 projektos(esp-radiator, esp-server, esp-web). Izstrādātais kods ir apskatāms GitHub repozitorijā[4].

Secinājumi

- Radiatora kontroles mehānismam nepieciešama rūpīga izpēte. Ideālam risinājumam jāspēj turēties pie radiatora un ar lineāru aktuatoru spiest uz vārstu dažādās precizitātēs.

- Risinājums nav universāls, jo radiatoriem atšķiras vītnes griezums, taču to var vienkārši pielāgot.
- ESP ļauj hostēt tīmekļa lietotni gan uz mājas rūtera interneta, gan kā pats savs rūteris kuram jāpieslēdzas, bet abos gadījumos tas neatļauj kontrolēt temperatūru no liela attāluma, piemēram, citas pilsētas. Tātad, ja ir nepieciešama iespēja mainīt temperatūru, kad nav mājās, tad ir nepieciešams risinājums, kurā sistēma un tīmekļa lietotne būtu pieejama no interneta.
- Vadu komunikācijā starp ESP motēm ir nepieciešams zemējuma (GND) vada savienojums starp motēm. Bez zemējuma savienojuma būs traucēta komunikācija ar skaņu.
- Vietējos veikalos detaļas ir dārgākas nekā pasūtot no Ķīnas lielākos daudzumos, bet pasūtot no Ķīnas jāgaida vismaz divas nedēļas. Ir arī iespējams aizņemties detaļas no DF Lab, bet par šo iespēju uzzinājām, kad viss jau bija iegādāts.
- 3D modelēšana ir sarežģīts process, kas ietver sevī rūpīgu produkta analīzi un izmēru pielāgošanu reālās dzīves izmantošanai.
- Nepieciešamos 3D modeļus var izdrukāt DF Lab par brīvu zem noteikta svara un modeļi šādos izmēros ir viegli.

Atsauces

[1] AVATTO viedie termostati - https://avatto-smart.com/product/smart-thermostat-radiator-valve-trv168/

[2] tado viedie termostati - https://shop.tado.com/en-row

[3] Termostata uzbūve - https://www.cibsejournal.com/technical/balancing-act-pressure-independent-thermostatic-radiator-valves/

[4] Projekta GitHub repozitorijs - https://github.com/Richard0exe/BSP-SMART-RADIATOR-THERMOSTAT